

УНИВЕРЗИТЕТ „СВ.КИРИЛ И МЕТОДИЈ“ СКОПЈЕ

ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО



Елаборат за проектна задача

Calorie Tracker

Предмет	Континуирана интеграција и испорака
Професор	вонр. Проф. Д-р Панче Рибарски
Студент	Душица Велкова
Број на индекс	233166

Септември, 2026

1. Краток опис на проектот

„Calorie Tracker“ е веб-апликација наменета за следење и управување со дневниот внес на храна, калории и макронутриенти. Корисникот може да внесува оброци, да избира храна од базата и да ја следи количината на внесени калории.

Апликацијата е реализирана како систем составен од frontend, backend и PostgreSQL база на податоци. Frontend-от го обезбедува корисничкиот интерфејс, backend-от ја обработува логиката и API барањата, а базата ги складира податоците.

Проектот е контејнеризиран со Docker и Docker Compose, а конфигуриран е и за работа во локален Kubernetes кластер. За автоматизација на градењето и објавувањето на Docker images се користи GitHub Actions, со нивно објавување на Docker Hub.

2. Функционалности на апликацијата

Апликацијата „Calorie Tracker“ овозможува едноставно следење на дневниот внес на храна и калории. Корисникот може да внесува различни оброци и да ги следи нивните нутритивни вредности.

Главните функционалности на апликацијата се:

- **Додавање оброк** – корисникот може да додаде нов оброк.
- **Избор на храна** – достапна е листа на однапред дефинирани намирници од PostgreSQL базата на податоци.
- **Внесување количина и единица** – при внесување на храна може да се дефинира количината и мерната единица.
- **Пресметување калории** – апликацијата ги прикажува калориите за внесената храна и количина.
- **Следење на макронутриенти** – за оброците се прикажуваат протеини, јаглехидрати и масти.

- **Категоризација на оброци** – оброците може да се организираат според тип, како појадок, ручек, вечера и ужина.
- **Бришење оброк** – корисникот може да отстрани оброк од листата.
- **Дневна калориска цел** – корисникот може да постави сопствена дневна калориска цел и да го следи внесот во однос на неа.

Податоците за храната и внесените оброци се зачувуваат во PostgreSQL базата на податоци, додека frontend делот комуницира со backend преку REST API.

Слика 1. Главен кориснички интерфејс на апликацијата

3. Технологии и алатки

За развој, контејнеризација и поставување на апликацијата „Calorie Tracker“ се користени повеќе технологии и алатки.

Frontend:

За изработка на корисничкиот интерфејс се користени HTML, CSS и JavaScript. Frontend апликацијата се сервира преку Nginx.

Backend:

Backend делот е реализиран со Node.js и Express.js. Тој е одговорен за обработка на барањата од frontend-от и комуникацијата со базата на податоци преку REST API.

База на податоци:

Како систем за управување со базата на податоци се користи PostgreSQL. Во базата се чуваат податоците за храната и внесените оброци.

Docker:

Docker се користи за контејнеризација на frontend и backend сервисите. Со Docker Compose сервисите и PostgreSQL базата може да се стартуваат и управуваат како една целина.

Kubernetes:

Kubernetes се користи за оркестрација и управување со контејнерите. Во проектот се користат Deployments, Services, Ingress, StatefulSet, PersistentVolumeClaim, ConfigMap и Secret.

GitHub и GitHub Actions:

GitHub се користи за верзионирање и складирање на изворниот код. GitHub Actions се користи за автоматизирање на процесот на градење и објавување на Docker images.

Docker Hub:

Docker Hub се користи како registry за складирање и објавување на backend и frontend Docker images.

4. Архитектура на системот

Апликацијата „Calorie Tracker“ е организирана како систем од три главни сервиси кои имаат различни улоги. Корисникот комуницира со frontend-от, frontend-от ги испраќа барањата до backend-от преку REST API, а backend-от ги обработува податоците и комуницира со PostgreSQL базата.

Компонента	Улога
Frontend	Прикажување на корисничкиот интерфејс и комуникација со API
Backend	Обработка на барања и имплементација на REST API
PostgreSQL	Складирање на податоците за храна и оброци

Комуникацијата помеѓу компонентите е:

Корисник → Frontend → Backend → PostgreSQL

Системот е дополнително контејнеризиран со Docker, додека Kubernetes овозможува управување со сервисите и нивното поврзување во deployment околина.

5. Docker u Docker Compose

Со docker compose up --build се стартуваат трите сервиси заедно. Frontend-от е достапен на <http://localhost:8080>, а backend-от на портата **5000**. PostgreSQL користи volume за перзистентно зачувување на податоците, додека сервисите комуницираат преку внатрешната Docker мрежа.

Контејнер	Image	Пристап
calorie_frontend	се гради од Dockerfile	localhost:8080 → frontend:80
calorie_backend	се гради од Dockerfile	localhost:5000
calorie_db	postgres:15-alpine	volume: db_data

Променливите POSTGRES_USER, POSTGRES_PASSWORD и POSTGRES_DB се дефинирани во .env фајл, кој не се зачувува во Git репозиториумот.

```
PS C:\Users\Dusica\OneDrive\Desktop\calorie-tracker> docker compose ps 2>$null
NAME                IMAGE                COMMAND              SERVICE    CREATED        STATUS        PORTS
calorie_backend      calorie-tracker-backend "docker-entrypoint.s..." backend    About an hour ago Up About an hour 0.0.0.0:5000->5000/tcp, [::]:5000->
calorie_db           postgres:15-alpine    "docker-entrypoint.s..." db         20 hours ago   Up 20 hours    0.0.0.0:5432->5432/tcp, [::]:5432->
calorie_frontend     calorie-tracker-frontend "/docker-entrypoint..." frontend  About an hour ago Up About an hour 0.0.0.0:8080->8080/tcp, [::]:8080->80
/tcp
```

Слика 2. Стартувани Docker контејнери со Docker Compose

6. Континуирана интеграција со GitHub Actions

За автоматизација на процесот на градење на Docker image-и е користен **GitHub Actions**. Workflow-от се активира при секој push на master гранката.

Во рамки на workflow-от се извршуваат следните чекори:

1. Преземање на кодот од GitHub repository.
2. Најавување на Docker Hub преку GitHub Secrets.
3. Градење на Docker image за backend-от.
4. Градење на Docker image за frontend-от.
5. Објавување на image-ите на Docker Hub.

На овој начин, по секоја промена во проектот автоматски се креираат и објавуваат нови верзии на Docker image-ите.

Name	Last Pushed ↑	Contains	Visibility	Scout
dushicavellkova/calorie-frontend	about 2 hours ago	IMAGE	Public	Inactive
dushicavellkova/calorie-backend	about 2 hours ago	IMAGE	Public	Inactive

Слика 3. Backend и frontend Docker image-и објавени на Docker Hub Registry

7. Kubernetes

За deployment на апликацијата е користен Kubernetes, при што frontend-от, backend-от и PostgreSQL базата се извршуваат како одделни ресурси. Податоците се зачувуваат преку persistent storage, а пристапот до апликацијата е овозможен преку Ingress.

Компонента	Реализација во проектот
Frontend и Backend	Се извршуваат преку Kubernetes Deployments со по 2 реплики
PostgreSQL	Се извршува преку StatefulSet со persistent storage
Комуникација	Frontend и backend комуницираат преку Kubernetes Services
Надворешен пристап	Апликацијата е достапна преку Ingress на calorie-tracker.local
Конфигурација и податоци	Се користат ConfigMap и Secret за конфигурациски вредности и чувствителни податоци

```
PS C:\Users\Dusica\OneDrive\Desktop\calorie-tracker> kubectl get all -n calorie-tracker
NAME                                READY   STATUS    RESTARTS   AGE
pod/calorie-backend-7bd4854c49-6fgrx 1/1     Running   0           45h
pod/calorie-backend-7bd4854c49-rtldwh 1/1     Running   0           45h
pod/calorie-db-0                       1/1     Running   0           45h
pod/calorie-frontend-7986d7dc45-6ncq4 1/1     Running   0           45h
pod/calorie-frontend-7986d7dc45-d4nqf 1/1     Running   0           45h

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP   PORT(S)    AGE
service/backend-service             ClusterIP     10.43.30.130  <none>        5000/TCP   45h
service/db-service                  ClusterIP     10.43.194.79  <none>        5432/TCP   45h
service/frontend-service             ClusterIP     10.43.154.71  <none>        80/TCP     45h

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/calorie-backend     2/2     2             2           45h
deployment.apps/calorie-frontend     2/2     2             2           45h

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/calorie-backend-54f9d65db6 0         0         0       45h
replicaset.apps/calorie-backend-7bd4854c49 2         2         2       45h
replicaset.apps/calorie-frontend-5fd46df959 0         0         0       45h
replicaset.apps/calorie-frontend-7986d7dc45 2         2         2       45h

NAME                                READY   AGE
statefulset.apps/calorie-db          1/1     45h
```

Слика 4. Приказ на Kubernetes ресурсите и статусот на апликацијата

8. Постапување и пристап до апликацијата

Апликацијата може да се стартува локално на два начини.

8.1 Со Docker Compose:

```
docker compose up --build
```

По стартувањето, frontend-от е достапен на <http://localhost:8080>.

8.2 Со Kubernetes:

```
k3d cluster create calorie-cluster
```

```
kubectl apply -f k8s/
```

```
kubectl get all -n calorie-tracker
```

По успешно deploy-ирање, апликацијата е достапна преку локалниот Ingress на:

<http://calorie-tracker.local>

За локалниот домен е додаден соодветен запис во hosts фајлот.

9. Линкови

GitHub repository:

<https://github.com/dushicavelkova/Calorie-tracker>

Docker Hub – Backend:

<https://hub.docker.com/r/dushicavelkova/calorie-backend>

Docker Hub – Frontend:

<https://hub.docker.com/r/dushicavelkova/calorie-frontend>

GitHub Actions:

<https://github.com/dushicavelkova/Calorie-tracker/actions>

10. Алатки

Користени алатки: HTML, CSS, JavaScript, Node.js, Express.js, PostgreSQL, Docker, Docker Compose, Kubernetes, k3d, Nginx, Git, GitHub, GitHub Actions и Docker Hub.

11. Заклучок

Со изработката на „Calorie Tracker“ е реализирана веб-апликација за следење на внесот на храна, калории и макронутриенти. Апликацијата е реализирана со frontend, backend и PostgreSQL база, а нејзиното извршување и deployment се овозможени преку Docker, Docker Compose и Kubernetes.

Дополнително, со користење на GitHub Actions е автоматизиран процесот на градење и објавување на Docker image-ите на Docker Hub. Проектот овозможи практична примена на концепти поврзани со развој на веб-апликации, контејнеризација, CI и оркестрација со Kubernetes.